



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	Árboles
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero “B”
Alumnos participantes:	Moyolema Criollo Katherine Lissette
Asignatura:	Estructura de datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Desarrollar habilidades en el trabajo con árboles

Específicos:

- Implementar funciones recursivas para contar nodos en árboles N-arios y calcular la altura en árboles binarios
- Construir un Árbol Binario de Búsqueda insertando nodos, colocando menores a la izquierda y mayores a la derecha
- Realizar recorrido In-Order para obtener valores en orden ascendente y transformar un árbol binario en su imagen espejo intercambiando ramas.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 8

No presenciales: 0

2.4 Instrucciones

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC PC de escritorio o portatil con JDK instalado e IDE de su elección.

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☐ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☐ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): _____

2.6 Actividades por desarrollar



Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

2.7 Resultados obtenidos

Se desarrollan habilidades en la solución a un problema de baja complejidad utilizando árboles.

INFORME

1) Introducción

Los árboles son estructuras de datos jerárquicas fundamentales en computación, utilizadas para modelar relaciones como organigramas, sistemas de archivos y bases de datos. El dominio de sus operaciones básicas es esencial para comprender algoritmos más avanzados y optimizar la gestión de información.

El presente informe documenta la implementación de cinco ejercicios prácticos sobre árboles N-arios y árboles binarios, desarrollados en los lenguajes C++ y Java. Los ejercicios abordan el conteo de nodos en árboles N-arios, la inserción en un Árbol Binario de Búsqueda (BST), el cálculo de la altura de un árbol binario, el recorrido In-Order y la inversión de un árbol binario (árbol espejo). Cada algoritmo fue implementado de forma recursiva y probado con casos de prueba predefinidos, verificando que los resultados obtenidos coincidan con los esperados [1][2].

2) Marco teórico

2.1) Árboles como estructura de datos

Un árbol es una estructura de datos no lineal que organiza los elementos en una jerarquía. Está compuesto por nodos conectados entre sí mediante aristas, donde existe un nodo especial llamado raíz que no tiene padre, y el resto de nodos se organizan en niveles descendentes. A diferencia de los grafos, los árboles no contienen ciclos [1][2].

2.2) Árboles N-arios

Un árbol N-ario es aquel en el que cada nodo puede tener un número variable de hijos, sin un límite máximo predefinido. Cada nodo contiene un valor y una lista o vector que almacena las referencias a sus nodos hijos. Este tipo de árbol es útil para representar estructuras como directorios de archivos o jerarquías organizacionales [2][5].

2.3) Árboles binarios

Un árbol binario es un caso particular de árbol donde cada nodo puede tener como máximo dos hijos, denominados hijo izquierdo e hijo derecho. Esta restricción simplifica muchas operaciones y permite implementar estructuras más especializadas [1][3].

2.4) Árbol Binario de Búsqueda (BST)

Un Árbol Binario de Búsqueda es un árbol binario que cumple con la siguiente propiedad: para cualquier nodo, todos los valores en su subárbol izquierdo son menores que el valor del nodo, y todos los valores en su subárbol derecho son mayores o iguales. Esta propiedad permite realizar búsquedas eficientes con complejidad $O(\log n)$ en promedio [1][4].

2.5) Altura de un árbol



La altura de un árbol se define como el número máximo de aristas desde la raíz hasta la hoja más profunda. Una altura pequeña indica un árbol equilibrado, mientras que una altura grande puede implicar un árbol degenerado, similar a una lista enlazada [3].

2.6) Recorrido In-Order

El recorrido In-Order es una técnica de exploración de árboles binarios que sigue la secuencia: subárbol izquierdo, nodo raíz, subárbol derecho. En un BST, este recorrido produce los valores ordenados de menor a mayor. Su implementación típica es recursiva [1][4].

2.7) Inversión de árbol (Árbol espejo)

La inversión de un árbol binario consiste en intercambiar recursivamente los subárboles izquierdo y derecho de cada nodo. El resultado es una imagen especular del árbol original. Esta transformación es útil en ciertos problemas de manipulación de estructuras jerárquicas [5].

2.8) Recursividad en árboles

Los árboles tienen una naturaleza recursiva: cada nodo puede considerarse como la raíz de un subárbol. Por esta razón, los algoritmos que operan sobre árboles suelen implementarse de forma recursiva, lo que simplifica el código y lo hace más legible [3].

3) Problema práctico

En el desarrollo de software, es común encontrarse con situaciones donde los datos deben organizarse de manera jerárquica. Por ejemplo, un sistema de archivos de una computadora tiene carpetas que contienen subcarpetas y archivos; el organigrama de una empresa muestra empleados que reportan a otros; o un árbol genealógico representa relaciones de parentesco entre personas.

Para manejar este tipo de estructuras, se necesitan herramientas que permitan almacenar, recorrer y transformar la información de forma eficiente. Los árboles N-arios y binarios son estructuras de datos diseñadas específicamente para resolver este tipo de problemas.

En esta práctica, se abordan cinco problemas prácticos fundamentales:

1. **Conteo de elementos:** Saber cuántos nodos tiene un árbol es útil para calcular tamaño de carpetas o cantidad de empleados en una organización.
2. **Organización de datos:** Los Árboles Binarios de Búsqueda (BST) permiten insertar y buscar información rápidamente, como en una base de datos o un diccionario.
3. **Medición de profundidad:** Conocer la altura de un árbol ayuda a evaluar su eficiencia y detectar si está desbalanceado.
4. **Recorrido ordenado:** Extraer los datos en orden ascendente es útil para generar reportes o listados ordenados automáticamente.
5. **Transformación de estructuras:** Invertir un árbol permite obtener su imagen espejo, lo cual puede ser necesario para ciertos algoritmos de procesamiento de imágenes o manipulación de estructuras simétricas.

Dominar estas operaciones básicas sobre árboles permite resolver problemas más complejos en áreas como bases de datos, compiladores, inteligencia artificial y sistemas operativos.



4) Desarrollo

4.1) Ejercicio 1: Conteo de nodos en un árbol N-ario

Descripción del problema

Dado un árbol N-ario, donde cada nodo puede tener cualquier cantidad de hijos, se debe implementar una función recursiva que cuente el número total de nodos presentes en el árbol.

Algoritmo implementado

El algoritmo sigue un enfoque recursivo de recorrido en profundidad (DFS):

1. Si el nodo actual es nulo, se retorna 0.
2. Se inicializa un contador en 1 para contar el nodo actual.
3. Se recorre cada uno de los hijos del nodo actual.
4. Por cada hijo, se suma recursivamente la cantidad de nodos de ese subárbol.
5. Se retorna el total acumulado.

Complejidad computacional

- **Complejidad temporal:** $O(n)$, donde n es el número total de nodos.
- **Complejidad espacial:** $O(h)$, donde h es la altura del árbol.

Fragmento de código:

cpp

```
int contarNodos(NodoN* raiz) {  
    if (raiz == nullptr) return 0;  
    int count = 1;  
    for (NodoN* hijo : raiz->hijos) {  
        count += contarNodos(hijo);  
    }  
    return count;  
}
```

java

```
public static int contarNodos(NodoN raiz) {  
    if (raiz == null) return 0;  
    int count = 1;  
    for (NodoN hijo : raiz.hijos) {  
        count += contarNodos(hijo);  
    }  
    return count;  
}
```

4.2) Ejercicio 2: Inserción en un Árbol Binario de Búsqueda (BST)

Descripción del problema

Se debe implementar la operación de inserción en un Árbol Binario de Búsqueda, respetando la propiedad fundamental: los valores menores se insertan en el subárbol izquierdo, mientras que los valores mayores o iguales se insertan en el subárbol derecho.

Algoritmo implementado

1. Si el nodo actual es nulo, se crea y retorna un nuevo nodo con el valor dado.
2. Si el valor a insertar es menor que el valor del nodo actual, se inserta recursivamente en el subárbol izquierdo.
3. Si el valor es mayor o igual, se inserta recursivamente en el subárbol derecho.
4. Se retorna el nodo actual (para mantener la estructura del árbol).

Complejidad computacional

- **Complejidad temporal promedio:** $O(\log n)$
- **Complejidad temporal en el peor caso:** $O(n)$
- **Complejidad espacial:** $O(h)$



```
java

public static int calcularAltura(Nodo raiz) {
    if (raiz == null) return 0;
    return 1 + Math.max(calcularAltura(raiz.izquierdo),
                        calcularAltura(raiz.derecho));
}
```

4.4) Ejercicio 4: Recorrido In-Order

Descripción del problema

Se debe implementar el recorrido In-Order de un árbol binario, que visita los nodos en la secuencia: subárbol izquierdo, nodo raíz, subárbol derecho. En un BST, este recorrido produce los valores ordenados de forma ascendente.

Algoritmo implementado

1. Si el nodo actual es nulo, se retorna sin hacer nada.
2. Se recorre recursivamente el subárbol izquierdo.
3. Se agrega el valor del nodo actual al vector/lista de resultados.
4. Se recorre recursivamente el subárbol derecho.

Complejidad computacional

- **Complejidad temporal:** $O(n)$
- **Complejidad espacial:** $O(n)$ para almacenar el resultado, más $O(h)$ de la pila de recursión.

Fragmento de código:

```
cpp

void inOrderAux(Nodo* nodo, vector<int>& resultado) {
    if (nodo == nullptr) return;
    inOrderAux(nodo->izquierdo, resultado);
    resultado.push_back(nodo->valor);
    inOrderAux(nodo->derecho, resultado);
}

vector<int> recorridoInOrder(Nodo* raiz) {
    vector<int> resultado;
    inOrderAux(raiz, resultado);
    return resultado;
}
```

```
java

inOrderAux(nodo.izquierdo, resultado);
resultado.add(nodo.valor);
inOrderAux(nodo.derecho, resultado);
}

public static List<Integer> recorridoInOrder(Nodo raiz) {
    List<Integer> resultado = new ArrayList<>();
    inOrderAux(raiz, resultado);
    return resultado;
}
```



4.5) Ejercicio 5: Inversión de un árbol binario (Árbol espejo)

Descripción del problema

Se debe implementar una función que invierta un árbol binario, intercambiando recursivamente los subárboles izquierdo y derecho de cada nodo. El resultado es una imagen especular del árbol original.

Algoritmo implementado

1. Si el nodo actual es nulo, se retorna null.
2. Se invierte recursivamente el subárbol izquierdo.
3. Se invierte recursivamente el subárbol derecho.
4. Se intercambian los punteros izquierdo y derecho del nodo actual.
5. Se retorna el nodo actual.

Complejidad computacional

- **Complejidad temporal:** $O(n)$
- **Complejidad espacial:** $O(h)$

Fragmento de código:

cpp

```
Nodo* invertir(Nodo* raiz) {  
    if (raiz == nullptr) return nullptr;  
    Nodo* temp = raiz->izquierdo;  
    raiz->izquierdo = invertir(raiz->derecho);  
    raiz->derecho = invertir(temp);  
    return raiz;  
}
```

java

```
public static Nodo invertir(Nodo raiz) {  
    if (raiz == null) return null;  
    Nodo temp = raiz.izquierdo;  
    raiz.izquierdo = invertir(raiz.derecho);  
    raiz.derecho = invertir(temp);  
    return raiz;  
}
```

5) Resultados obtenidos

Ejercicio	Prueba	Esperado	Obtenido
1	Conteo de nodos	6	6
2	Inserción BST	10,5,15,3	10,5,15,3
3	Altura del árbol	3	3
4	Recorrido In-Order	1,2,3,4,5,6,7	1,2,3,4,5,6,7
5	Inversión de árbol	Izq:3, Der:2	Izq:3, Der:2

Observaciones importantes:

- **Ejercicio 1:** La función recursiva contó correctamente los 6 nodos del árbol N-ario, visitando cada nodo una sola vez.
- **Ejercicio 2:** Las inserciones respetaron la propiedad del BST. El valor 3 se ubicó correctamente como hijo izquierdo del nodo 5.



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



- **Ejercicio 3:** La altura se calculó correctamente considerando el camino más largo desde la raíz hasta la hoja más profunda ($1 \rightarrow 2 \rightarrow 3$). El caso base con árbol nulo retornó 0.
- **Ejercicio 4:** El recorrido In-Order produjo la secuencia ordenada ascendente [1,2,3,4,5,6,7], demostrando la propiedad fundamental de los BST.
- **Ejercicio 5:** La inversión intercambió correctamente los hijos izquierdo y derecho del nodo raíz, transformando [2,3] en [3,2].

6) Capturas de ejecución (C++ y Java)

• **Ejercicio 1:**

- **C++**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> g++ Ejercicio1_Basico.cpp
-o Ejercicio1_Basico
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> ./Ejercicio1_Basico
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
```

- **Java**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\java> javac Ejercicio1_Basico.java
PS C:\Users\Katherine\Downloads\APE_Arboles\java> java Ejercicio1_Basico
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
```

• **Ejercicio 2:**

- **C++**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> g++ Ejercicio2_Binario.cpp
-p -o Ejercicio2_Binario
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> ./Ejercicio2_Binario
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
Hijo Izquierdo (Esperado 5): 5
Hijo Derecho (Esperado 15): 15
Hijo Izq del 5 (Esperado 3): 3
```

- **Java**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\java> javac Ejercicio2_Binario.java
PS C:\Users\Katherine\Downloads\APE_Arboles\java> java Ejercicio2_Binario
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
Hijo Izquierdo de Raiz (Esperado 5): 5
Hijo Derecho de Raiz (Esperado 15): 15
Hijo Izquierdo del 5 (Esperado 3): 3
```

• **Ejercicio 3:**

- **C++**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> g++ Ejercicio3_Binario.cpp
-p -o Ejercicio3_Binario
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> ./Ejercicio3_Binario
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
Altura de arbol nulo (esperado 0): 0
```

- **Java**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\java> javac Ejercicio3_Binario2.java
PS C:\Users\Katherine\Downloads\APE_Arboles\java> java Ejercicio3_Binario2
--- Prueba Ejercicio 3 ---
Altura esperada: 3
Altura calculada: 3
Altura de árbol nulo (esperado 0): 0
```

• **Ejercicio 4:**



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: ENERO – JULIO 2026



- **C++**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> g++ Ejercicio4_Recorridos.cpp -o Ejercicio4_Recorridos
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> ./Ejercicio4_Recorridos
--- Prueba Ejercicio 4 ---
Resultado esperado: 1 2 3 4 5 6 7
Tu resultado:      1 2 3 4 5 6 7
```

- **Java**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\java> javac Ejercicio4_RecorridoInOrder.java
PS C:\Users\Katherine\Downloads\APE_Arboles\java> java Ejercicio4_RecorridoInOrder
--- Prueba Ejercicio 4 ---
Resultado esperado: [1, 2, 3, 4, 5, 6, 7]
Tu resultado:      [1, 2, 3, 4, 5, 6, 7]
```

• **Ejercicio 5:**

- **C++**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> g++ Ejercicio5_Transformacion.cpp -o Ejercicio5_Transformacion
PS C:\Users\Katherine\Downloads\APE_Arboles\cpp> ./Ejercicio5_Transformacion
--- Prueba Ejercicio 5 ---
Antes de invertir:
Hijo Izq: 2 | Hijo Der: 3

Después de invertir (Esperado: Izq 3 | Der 2):
Hijo Izq: 3 | Hijo Der: 2
```

- **Java**

```
PS C:\Users\Katherine\Downloads\APE_Arboles\java> javac Ejercicio5_Transformacion.java
PS C:\Users\Katherine\Downloads\APE_Arboles\java> java Ejercicio5_Transformacion
--- Prueba Ejercicio 5 ---
Antes de invertir:
Hijo Izq: 2 | Hijo Der: 3

Después de invertir (Esperado: Izq 3 | Der 2):
Hijo Izq: 3 | Hijo Der: 2
```

7) Estructura del repositorio

- **README.md**
- **informe.pdf**
- **Capturas/**
 - **C++/** → Ejercicio1.png, Ejercicio2.png, Ejercicio3.png, Ejercicio4.png, Ejercicio5.png
 - **Java/** → Ejercicio1.png, Ejercicio2.png, Ejercicio3.png, Ejercicio4.png, Ejercicio5.png
- **cpp/**
 - Ejercicio1_Basico.cpp
 - Ejercicio2_Binario.cpp
 - Ejercicio3_Binario.cpp
 - Ejercicio4_Recorridos.cpp
 - Ejercicio5_Transformacion.cpp
- **java/**
 - Ejercicio1_Basico.java
 - Ejercicio2_Binario.java
 - Ejercicio3_Binario2.java
 - Ejercicio4_RecorridoInOrder.java
 - Ejercicio5_Transformacion.java

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía



- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

Las estructuras dinámicas de árboles constituyen una alternativa al procesamiento secuencial de los datos, de amplia aplicación práctica.

- La implementación de funciones recursivas en ambos lenguajes permitió comprender de manera práctica la lógica de recorrido y conteo en estructuras jerárquicas, fortaleciendo la capacidad de abstracción.
- La construcción de Árboles Binarios de Búsqueda evidenció la importancia de aplicar correctamente las reglas de inserción, garantizando eficiencia en la organización y búsqueda de datos.
- Los recorridos In-Order y la transformación en imagen espejo demostraron la versatilidad de los árboles binarios para representar y manipular información en distintos contextos de programación.

2.10 Recomendaciones

Tener en cuenta las condiciones específicas de cada situación para decidir qué estructura de datos utilizar.

- Continuar practicando ejercicios de inserción, recorrido y transformación de árboles en Java y C++ para consolidar la lógica de programación y el dominio de estructuras dinámicas.
- Ampliar los casos de prueba con diferentes conjuntos de datos, de modo que se evidencien las ventajas y limitaciones de cada recorrido y operación sobre árboles.
- Integrar los conceptos aprendidos en proyectos aplicados (como sistemas web o bases de datos) para relacionar la teoría con problemas reales y fortalecer el aprendizaje significativo.

2.11 Referencias bibliográficas

Insertar las referencias bibliográficas empleadas aplicando la norma IEEE.

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.
- [2] L. Joyanes Aguilar, I. Zahonero, L. García Sánchez, and M. Fernández Azuela, *Estructura de Datos en C++*. México D.F., México: McGraw-Hill, 2007.
- [3] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 2nd ed. Boston, MA, USA: Addison Wesley, 2007.
- [4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison Wesley, 2011.
- [5] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA, USA: Addison Wesley, 1997.



2.12 Anexos

- **Link de github:** <https://github.com/kmoyolema3929/APE3.-ARBOLES.git>

- **Compilación y ejecución:**

- **C++:**

Requisitos: Tener instalado un compilador de C++ (g++, MinGW, o Dev-C++).

Paso a paso:

1. Abrir una terminal (CMD en Windows, Terminal en Linux/Mac).
2. Navegar hasta la carpeta donde se encuentra el archivo .cpp.
3. Ejecutar el siguiente comando para compilar:

```
```bash
g++ Ejercicio1_Basico.cpp -o Ejercicio1_Basico
```

4. Ejecutar programa compilado:

```
En Linux/Mac:
./Ejercicio1_Basico
```

```
En Windows:
Ejercicio1_Basico.exe
```

- **Java:**

**Requisitos:** Tener instalado el JDK (Java Development Kit) versión 8 o superior.

**Paso a paso:**

1. Abrir una terminal.
2. Navegar hasta la carpeta donde se encuentra el archivo .java.
3. Ejecutar el siguiente comando para compilar:

```
javac Ejercicio1_Basico.java
```

4. Ejecutar el programa compilado:

```
java Ejercicio1_Basico
```